

API JSON Home Assignment

S02 - TextFromImage-Url-HW:

[Japanese-Crush-Course.jpg \(1280×720\)](https://nihongonana.com/wp-content/uploads/2025/01/Japanese-Crush-Course.jpg)

URL: <https://nihongonana.com/wp-content/uploads/2025/01/Japanese-Crush-Course.jpg>

Go to PSTMAN and paste URL and generate.

The screenshot shows the Postman interface with a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is a JSON object with the following structure:

```
{
  "model": "meta-llama/llama-4-scout-17b-16e-instruct",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "This is a image. Extract and return only the text exactly as shown, preserving case."
        },
        {
          "type": "image_url",
          "image_url": {
            "url": "https://nihongonana.com/wp-content/uploads/2025/01/Japanese-Crush-Course.jpg"
          }
        }
      ]
    }
  ]
}
```

The response is a JSON object with the following structure:

```
{
  "index": 0,
  "message": {
    "role": "assistant",
    "content": "FIRST STEP!\nJAPANESE\nCRUSH\nCOURSE"
  },
  "logprobs": null,
  "finish_reason": "stop"
}
```

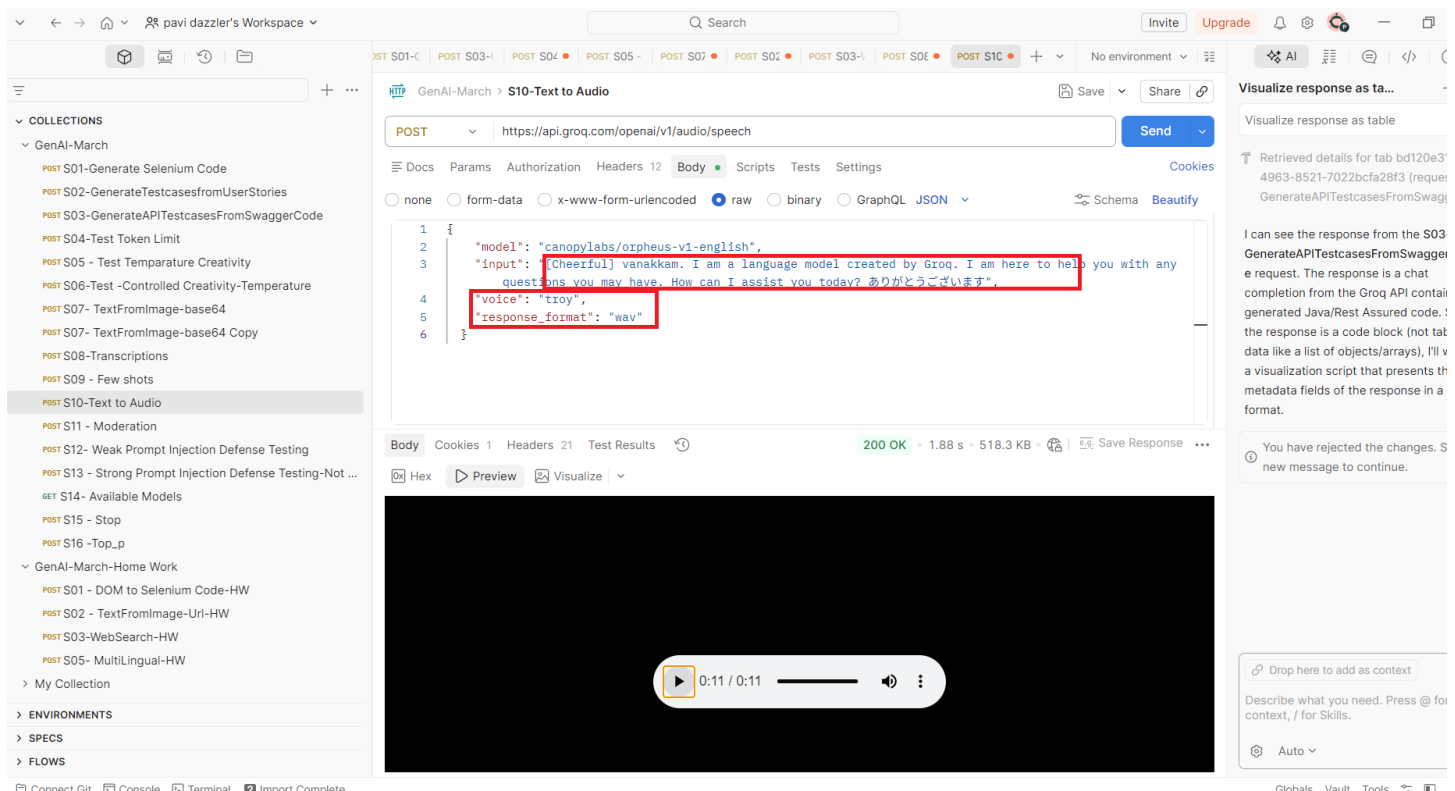
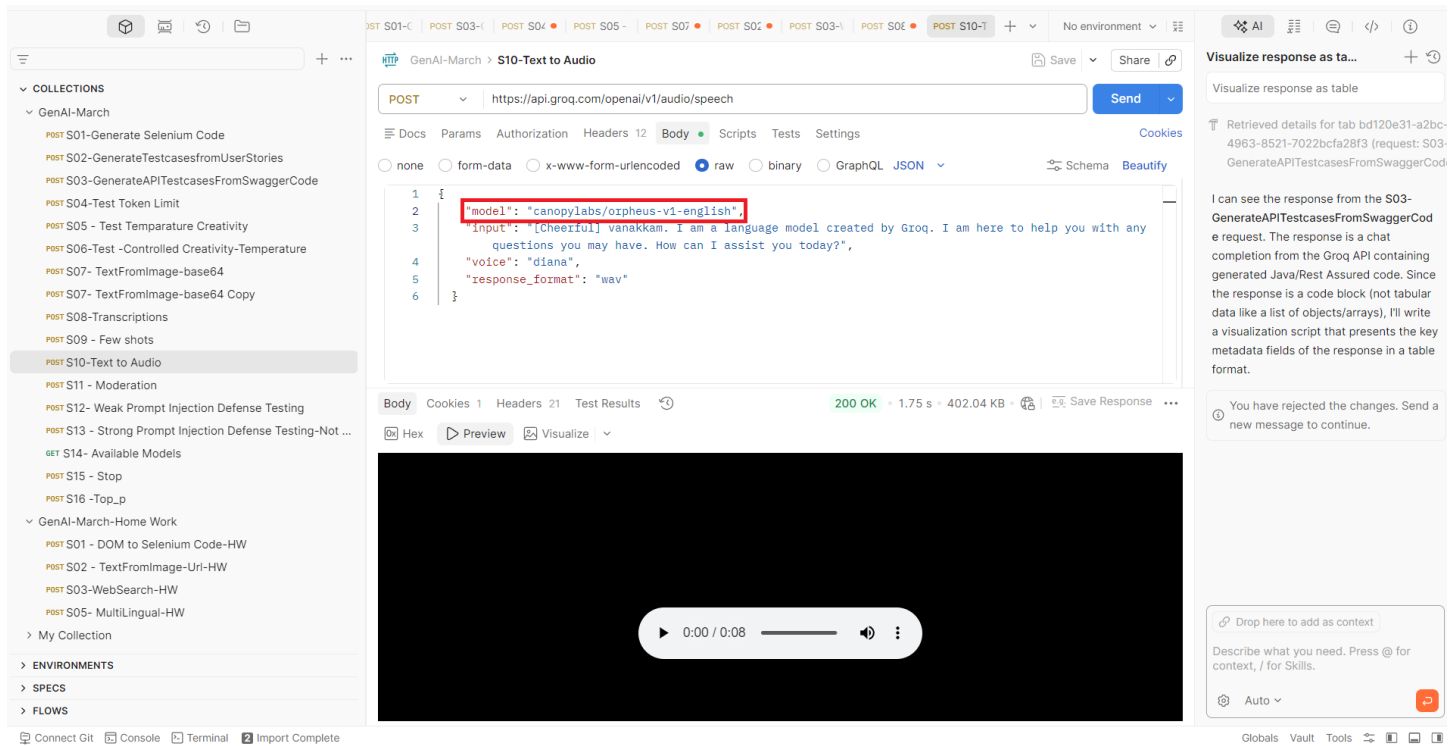
The response status is 200 OK. The right sidebar shows a message: "I can see the response from the S03-GenerateAPITestcasesFromSwaggerCode request. The response is a chat completion from the Groq API containing generated Java/Rest Assured code. Since the response is a code block (not tabular data like a list of objects/arrays), I'll write a visualization script that presents the key metadata fields of the response in a table format."

S10-Text to Audio

Go to groq: Click Text to Speech -> Orpheus English -> Copy URL

The screenshot shows the Groq Playground interface. The URL is `https://console.groq.com/playground?model=canopylabs/orpheus-v1-english`. The interface includes a "Text to Speech" section with a text input field and a "Submit" button. The top navigation bar includes "Playground", "API Keys", "Dashboard", and "Docs".

Go to Postman and paste over there:



Available voices with different tones:

<https://console.groq.com/docs/text-to-speech/orpheus#available-voices>

S01 - DOM to Selenium Code-HW

Go to any page: [Start to Speak Japanese in Minutes](#)
Copy DOM page element using “Edit as HTML”

The screenshot shows a web browser displaying the JapanesePod101 website. The page has a red header with the text "Learn Japanese. Thousands of lessons. No credit card needed." and a yellow "Join for Free" button. Below the header, there's a section titled "3 Reasons Why You Really Can Learn & Speak Japanese with JapanesePod101" with a sub-header "Short Audio & Video Lessons for Fast and Easy Learning". The main content area features a woman in a blue dress and a diagram showing a progression from "BEGINNER" to "ADVANCED". At the bottom, there's a section titled "Study Tools for Rapid Learning" with text about word lists, slideshows, and flashcards. The browser's DOM inspector is open on the right, showing the HTML structure of the page. The selected element is a div with class "glp--white". The DOM tree shows the following structure:

```
<div class="glp--white">
  <div class="glp-reasons glp--grey">
    <div class="glp-reason__container glp-reason__container--first">
      <div class="glp-reason__number">1</div>
      <div class="glp-reason__content">
        <h2 class="glp-reason__title">Short Audio & Video Lessons for Fast and Easy Learning</h2>
        <div class="glp-reason__image-box--mobile">
          
        </div>
      </div>
    </div>
  </div>
</div>
```

The Styles pane shows the following styles for the selected element:

```
element.style {
}
.glp--white {
  background-color: #fff;
}
div {
  display: block;
  unicode-bidi: isolate;
}
Inherited from body.responsive-b
.responsive-b {
  height: auto;
  margin: 0;
  padding: 0;
}
```

Go to gorq -> [GorqCloud](#) -> Playground -> Change model -> copy code -> Paste in JSON model

The screenshot shows the Groq Playground interface. The URL bar displays "https://console.groq.com/playground". The page has a header with "groq" and navigation links for "Personal", "Default Project", "Playground", "API Keys", "Dashboard", "Docs", and a user profile icon. The main area is divided into three sections: "SYSTEM" (with a text input for "Enter system message (Optional)"), "USER" (with a text input for "Enter user message..."), and a "curl" section. The "curl" section contains a curl command for a POST request to "https://api.groq.com/". A dropdown menu is open, showing a search bar "Search Models..." and a list of models. The selected model is "openai/gpt-oss-120b". The list of models includes:

- groq/compound-mini
- Meta
- llama-3.1-8b-instant
- llama-3.3-70b-versatile
- meta-llama/llama-4-scout-17b-16e-i...
- meta-llama/llama-prompt-guard-2-2...
- meta-llama/llama-prompt-guard-2-8...
- OpenAI
- openai/gpt-oss-120b (checked)
- openai/gpt-oss-20b
- openai/gpt-oss-safeguard-20b
- whisper-large-v3
- whisper-large-v3-turbo

At the bottom of the page, there are buttons for "+", "Clear", and "Submit Ctrl + ⏎".

S05- MultiLingual-HW

GenAI-March-Home Work > S05- MultiLingual-HW

POST S07 POST S02 POST S03- POST S06 POST S10 POST S01 - POST S12 GET http: POST S05-

No environment

Visualize response as ta...

Visualize response as table

Retrieved details for tab bd120e31-a2bc-4963-8521-7022bcfa28f3 (request: S03-GenerateAPITestcasesFromSwaggerCode)

I can see the response from the S03-GenerateAPITestcasesFromSwaggerCode request. The response is a chat completion from the Groq API containing generated Java/Rest Assured code. Since the response is a code block (not tabular data like a list of objects/arrays), I'll write a visualization script that presents the key metadata fields of the response in a table format.

You have rejected the changes. Send a new message to continue.

Drop here to add as context

Describe what you need. Press @ for context, / for Skills.

Auto

POST https://api.groq.com/openai/v1/chat/completions

Send

Docs Params Authorization Headers 10 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
2  "model": "openai/gpt-oss-120b",
3  "messages": [
4    {
5      "role": "system",
6      "content": "You are an API Specialist. Always respond in the same language as the user."
7    },
8    {
9      "role": "user",
10     "content": "பிரதமர் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்வதுங்கள்."
11   },
12   {
13     "role": "user",
14     "content": "ありがとうございます！このAPIの使い方を教えてください。"
15   }
16 ]
```

Body Cookies 1 Headers 20 Test Results 200 OK 3.63 s 4.54 KB Save Response

JSON Preview Visualize

```
8  "index": 0,
9  "message": {
10    "role": "assistant",
11    "content": "以下に、OpenAI API (ChatGPT) をご利用いただくための基本的な手順をご案内します。
\n\n---\n\n1 API キーの取得 \n1. **OpenAI アカウントにログイン** \n - https://platform.openai.com/ にアクセスし、アカウントでサインインします。 \n2. **API キーを作成** \n - ダッシュボード左側メニューの「API Keys」→「Create new secret key」をクリック。 \n - 作成されたシークレットキーは **一度だけ表示** されますので、必ず安全な場所に保存してください。 \n\n> **重要**：キーは他者と共有しないでください。コードや環境変数に埋め込む際は、Git などの公開リポジトリに含めないように注意してください。 \n\n---\n\n2 リクエストの基本構造 \n\n項目 | 説明 \n\n-----\n\n| \n | **エンドポイント** | https://api.openai.com/v1/chat/completions | \n | **HTTP メソッド** | POST | \n | **ヘッダー** | Authorization: Bearer YOUR_API_KEY | \n | **Content-Type** | application/json | \n | **リクエストボディ** | JSON 形式でモデル名やプロンプト、パラメータを指定します。 \n\n\n例: Python (requests) \n\nimport requests
import json
url = https://api.openai.com/v1/chat/completions
headers = {
  'Authorization': f'Bearer {api_key}',
  'Content-Type': 'application/json'
}
payload = {
  'model': 'gpt-4',
  'messages': [
    {'role': 'user', 'content': 'Hello!'}
  ]
}
response = requests.post(url, headers=headers, json=payload)
print(response.json())
```

Connect Git Console Terminal Import Complete

Globals Vault Tools

ENG 19:03